

iOS Seminar 2

Wafflestudio 2025



과제 0 공통 피드백

과제 0 공통 피드백

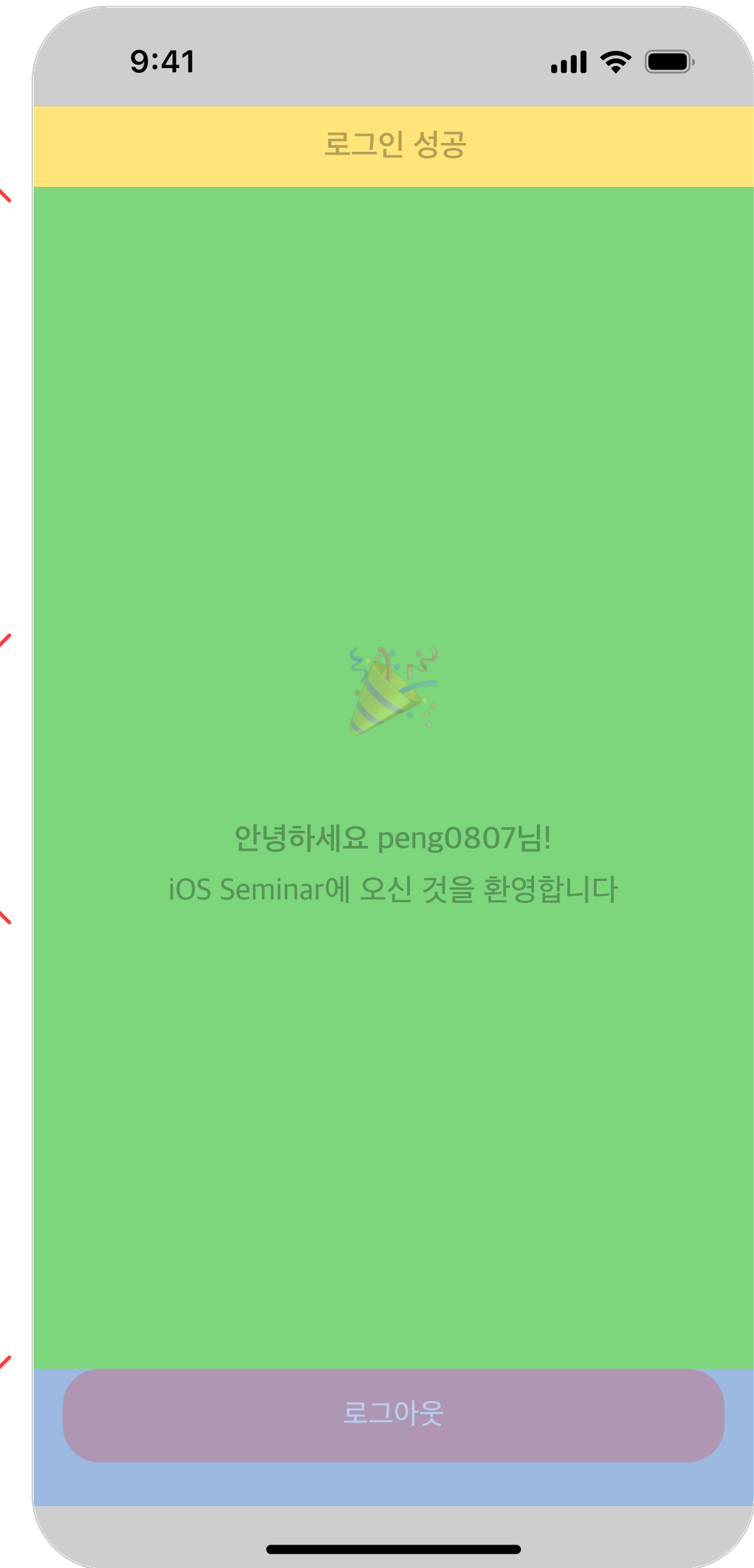
- 목표: 초록색 영역의 가운데에 View가 오도록 설계
- 오답 사례:
 - 초록+파랑 영역 전체의 가운데에 View 위치
 - 초록 영역에서 topAnchor 250.5로 설정
- 구현 예시:
 - 🎉 와 UILabel이 포함된 영역 생성 (UILayoutGuide)

Status Bar →
Navigation Bar →

250.5

250.5

Home Indicator →

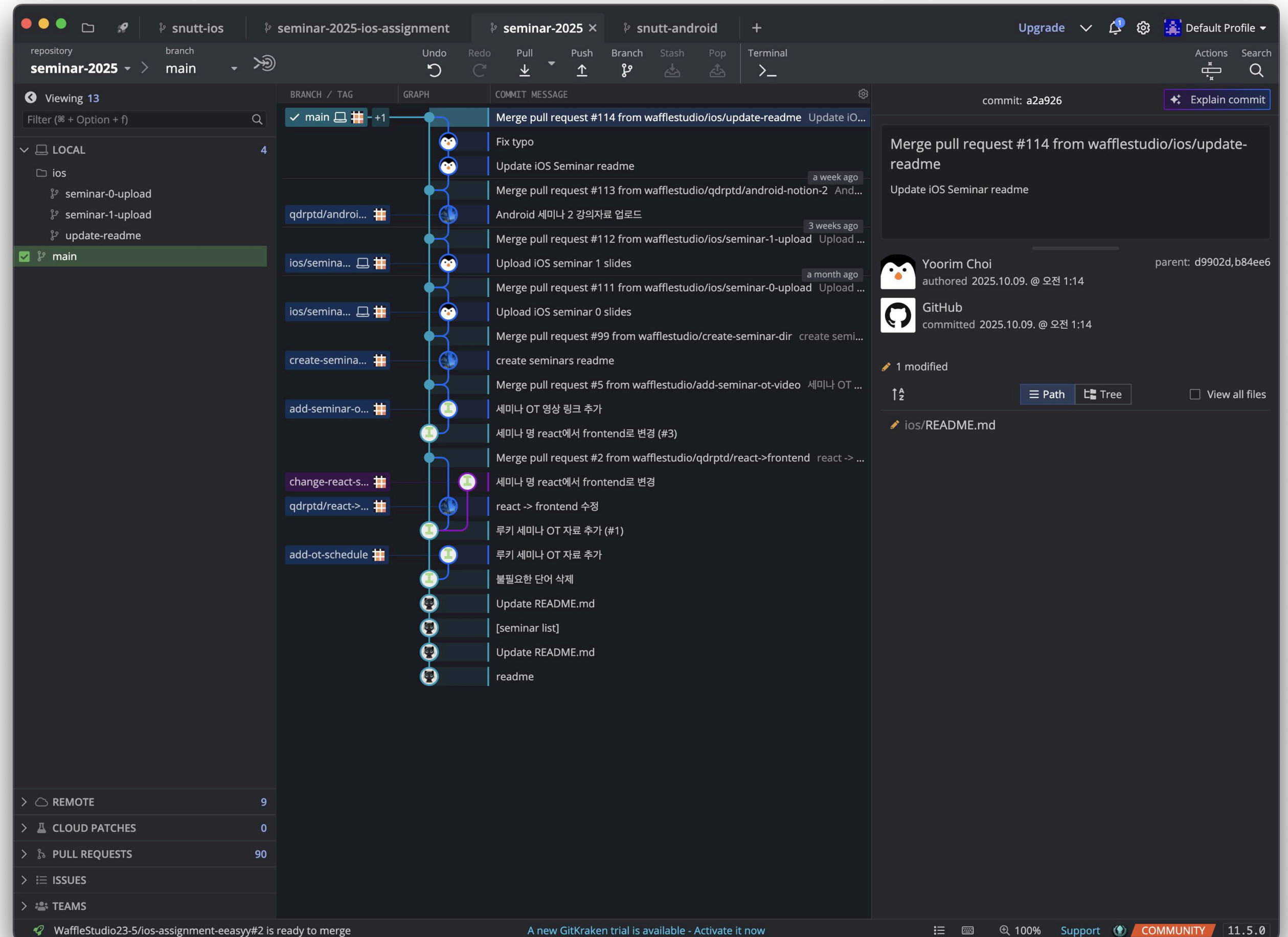


과제 0 공통 피드백

- Git 터미널이 익숙하지 않다면...

추천드리는 GUI 툴

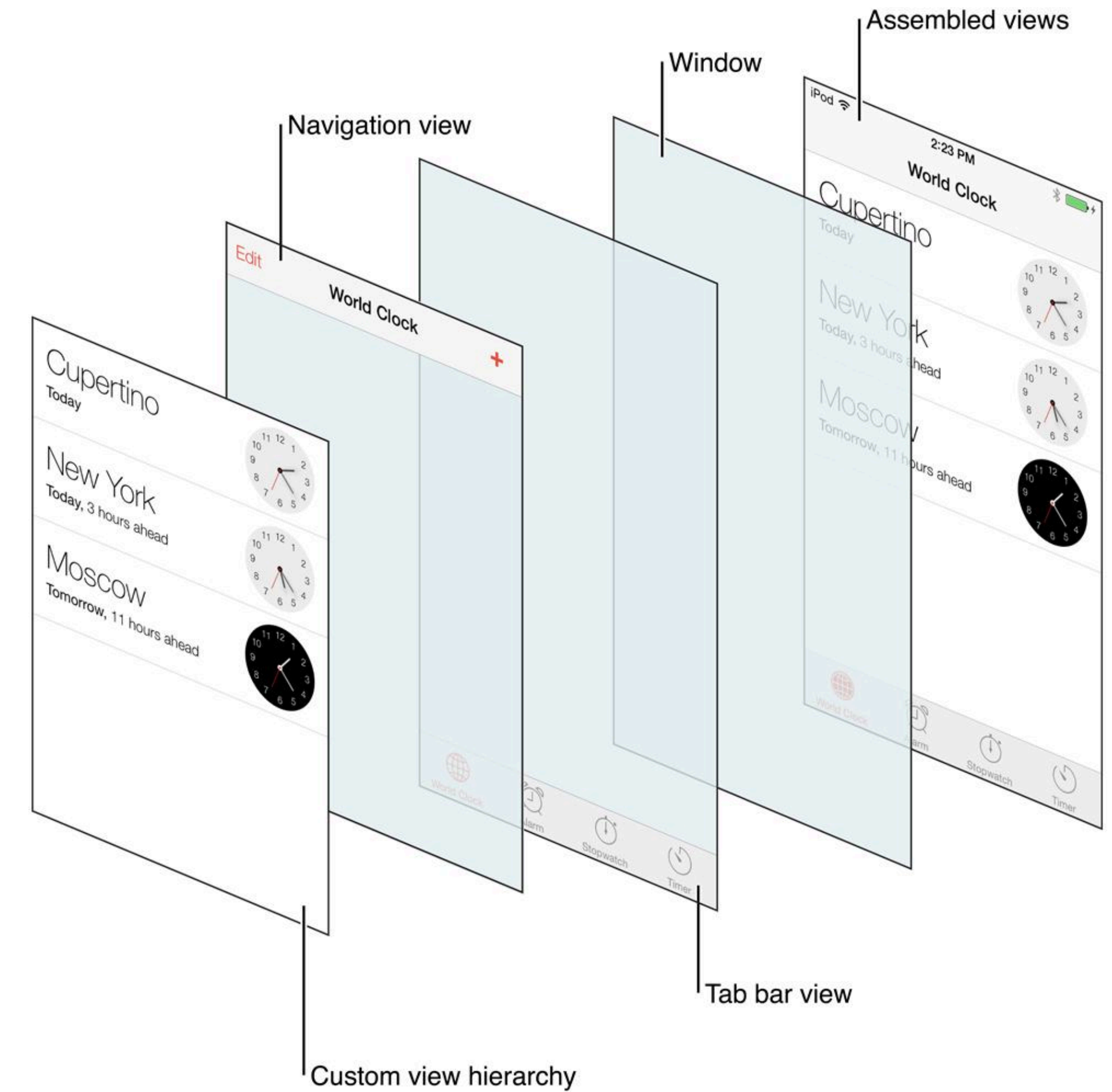
- GitKraken
- Github Desktop
- 구글에 “Git GUI 툴 추천” 입력



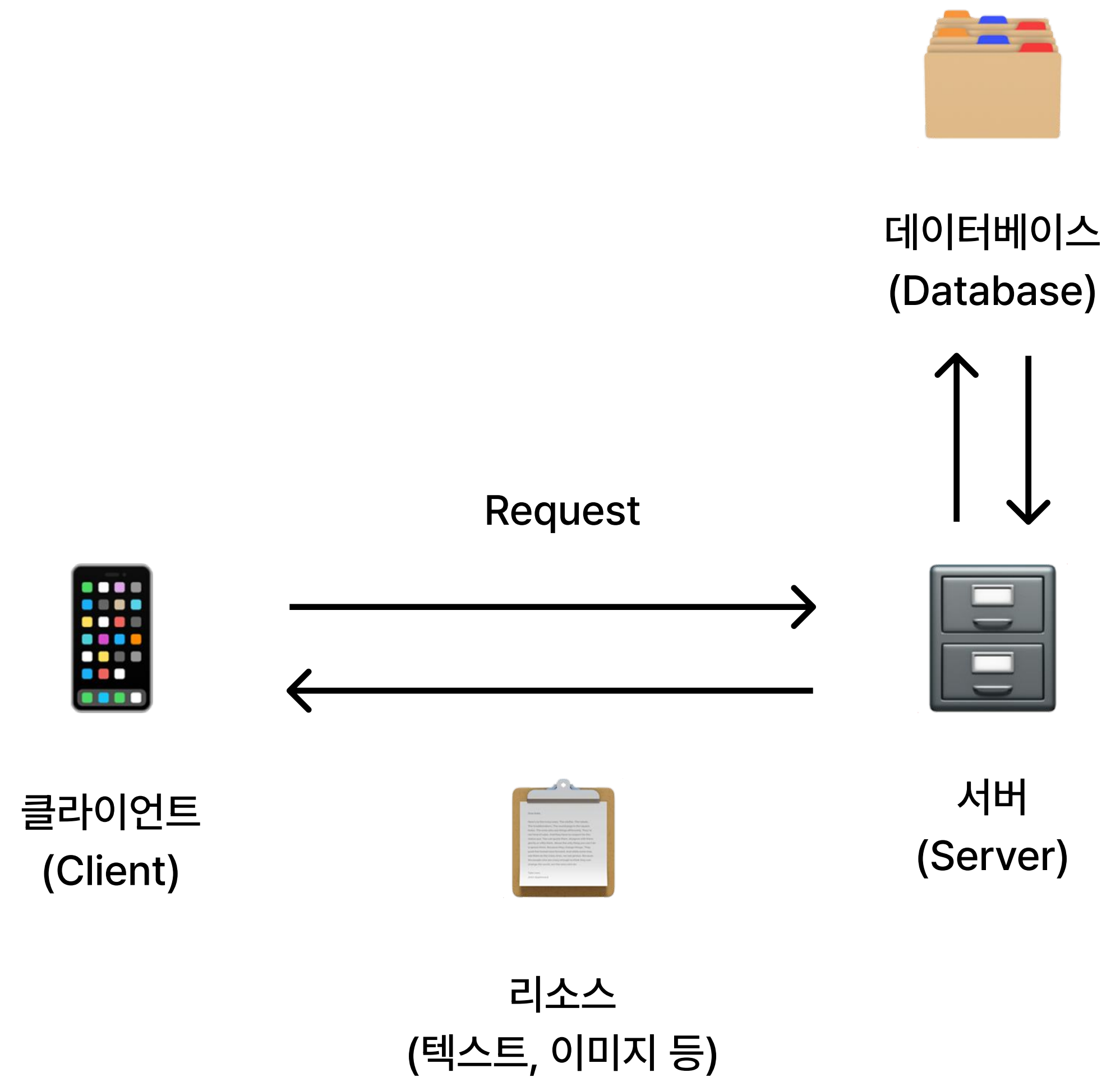
UITabBarController

UITabBarController (공식 문서)

- 여러 Child View Controller를 보여주기 위한 Container View Controller
- 각 탭은 별개의 ViewController를 root로 함
- 하단 Tab Bar 아이템을 커스터마이징하려면
UITabBarItem 사용
- UITabBarControllerDelegate를 상속하여 Tab Bar 인터페이스와 상호작용이 일어났을 때의 동작을 정의할 수 있음



Networking



REST API

- HTTP URI로 리소스 특정하고, HTTP Method로 리소스에 행할 동작을 정의
- 대표적인 메서드
 - **GET** : 리소스 조회
 - GET /user : 유저 정보 조회
 - **POST** : 리소스 생성
 - POST /user : 유저 생성
 - **PUT** : 기존 리소스 업데이트
 - PUT /user/password : 유저의 password 업데이트
 - **DELETE** : 리소스 삭제
 - DELETE /user : 유저 정보 삭제

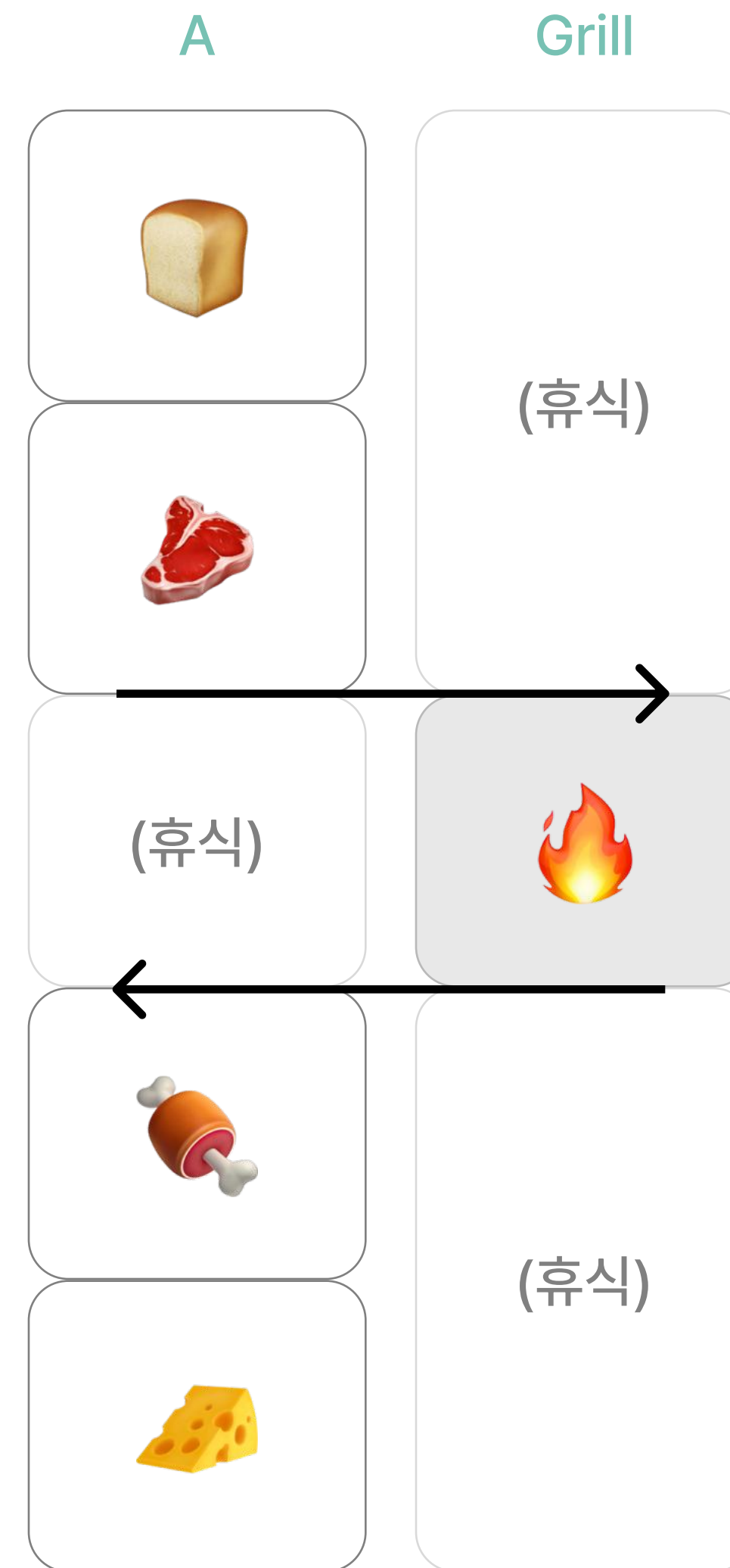
REST API

- Query Parameter
 - URL 뒤에 `?`와 함께 붙는 키-값(Key-Value) 쌍
 - GET `https://movie.com/user?id=wafflestudio&type=POPULAR`
- Path Variable
 - `{value}`로 특정 리소스 지칭
 - GET `https://movie.com/top100list`
- Request Body
 - POST 요청에 많이 사용되며, 일반적으로 JSON 형식

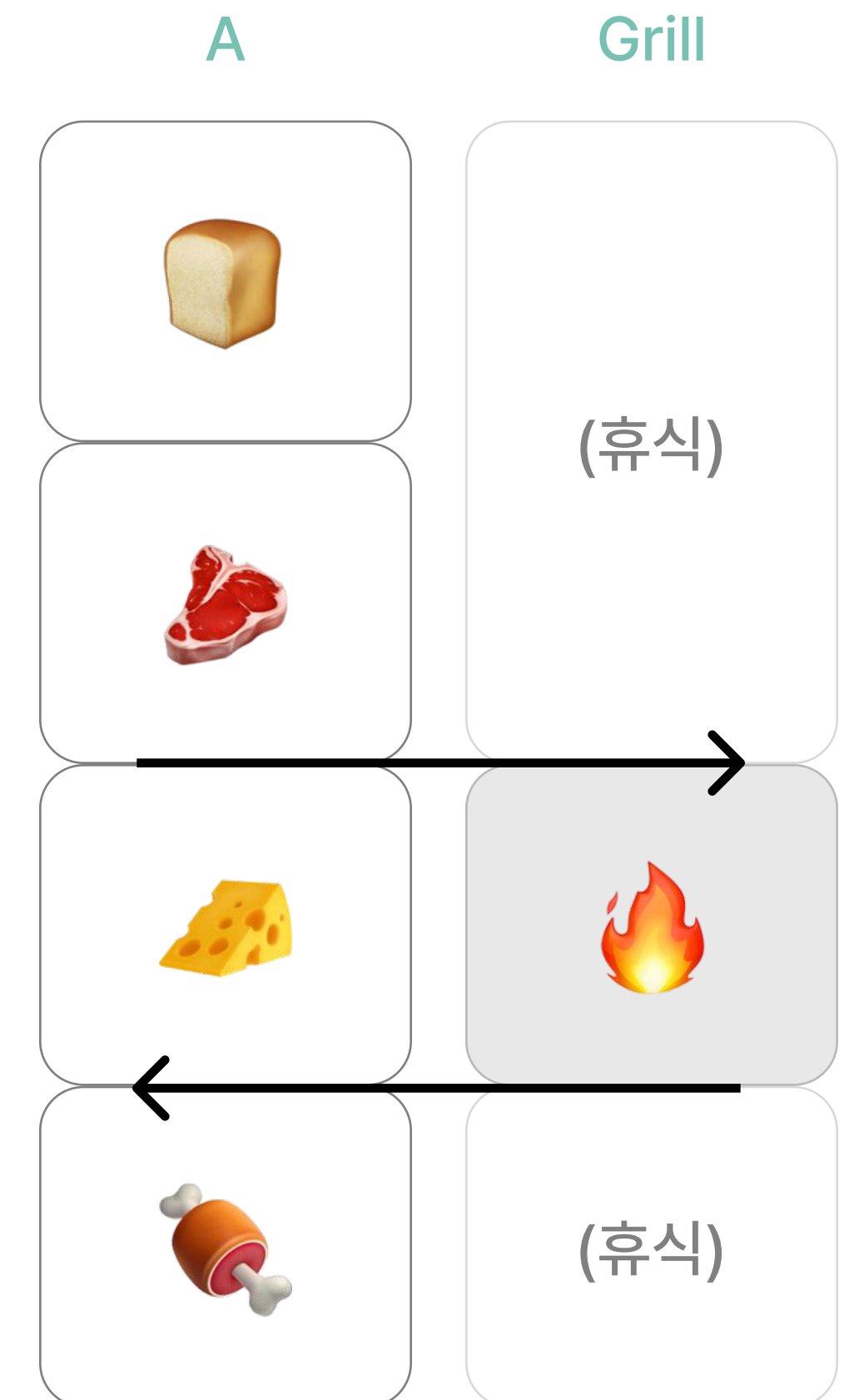
Concurrency

Asynchronous Code

- 🍔 햄버거 하나 만들어 줘
- 그릴에 고기가 구워지는 동안 다른 작업 진행
- cf. Synchronous Code



Synchronous 예시



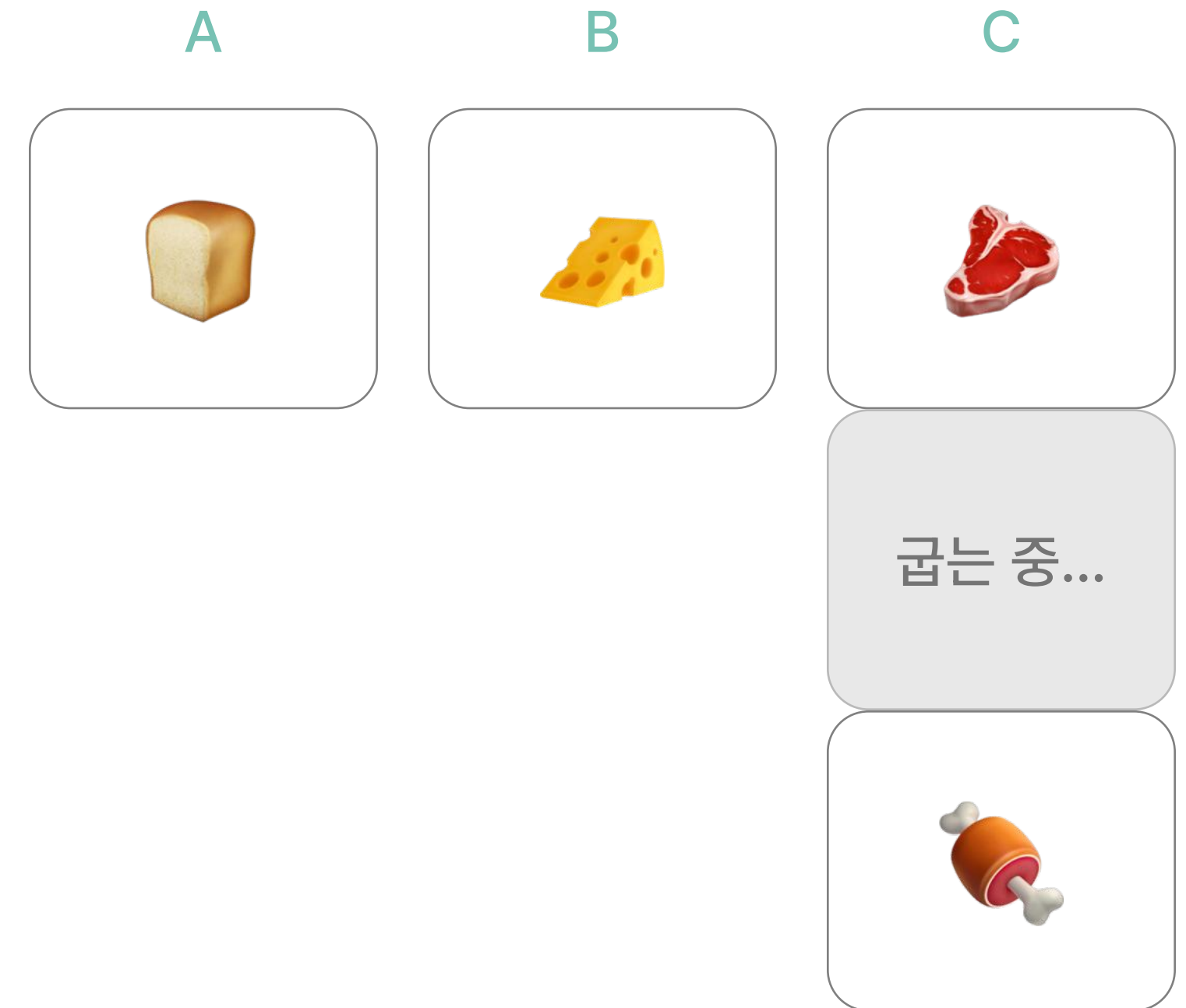
Asynchronous 예시

Parallel Code

- 🍔 햄버거 하나 만들어 줘
- 두 명 이상의 요리사가 역할을 분담하여 햄버거 만들기
- cf. Sequential Code



Sequential 예시



Parallel 예시

Swift Concurrency (공식 문서)

- Asynchronous하고 Parallel한 코드 작성
- Asynchronous Functions
 - `async` 키워드 추가: “이 함수/메서드는 실행 중간에 suspension될 만한 지점이 있다!”를 의미
 - `await` 키워드 추가: “suspension될 만한 지점”에 추가
 - (다른 asynchronous 메서드를 호출하는 경우에만 실제로 실행이 유예됨)

```
func loadPhoto(from photoNames: [String]) async -> [Photo] {  
    let firstPhoto = await downloadPhoto(named: photoNames[0]) // 대기 발생 (Async)  
    let secondPhoto = getCachedPhoto(named: photoNames[1]) // 대기 X (Sync)  
    let thirdPhoto = await downloadPhoto(named: photoNames[2]) // 대기 발생 (Async)  
  
    let photos = [firstPhoto, secondPhoto, thirdPhoto]  
}
```

Async Function 예시

Asynchronous Function+Parallel

- Parallel한 코드 작성: 동시에 진행되어도 괜찮은 작업과 그렇지 않은 작업 적절히 구분하기
 - `let + await`: 한 줄 씩 순서대로 진행+대기
 - `async-let + await`: `async-let`은 동시에 진행, 모두 완료될 때까지 `await`

```
let firstPhoto = await downloadPhoto(named: photoNames[0])
let secondPhoto = await downloadPhoto(named: photoNames[1])
let photos = [firstPhoto, secondPhoto]
```

`downloadPhoto()`



`downloadPhoto()`



`let photos = [firstPhoto, secondPhoto]`

let + await 예시

```
async let firstPhoto = downloadPhoto(named: photoNames[0])
async let secondPhoto = downloadPhoto(named: photoNames[1])
let photos = await [firstPhoto, secondPhoto]
```

`downloadPhoto()`

`downloadPhoto()`



`let photos = await [firstPhoto, secondPhoto]`

async let + await 예시

Task (공식 문서)

- asynchronous한 작업의 단위
- 하나의 Task는 한 번에 하나의 작업을 하지만, 여러 Task를 정의하면 Swift가 동시에 실행되도록 스케줄
- 예시 참고

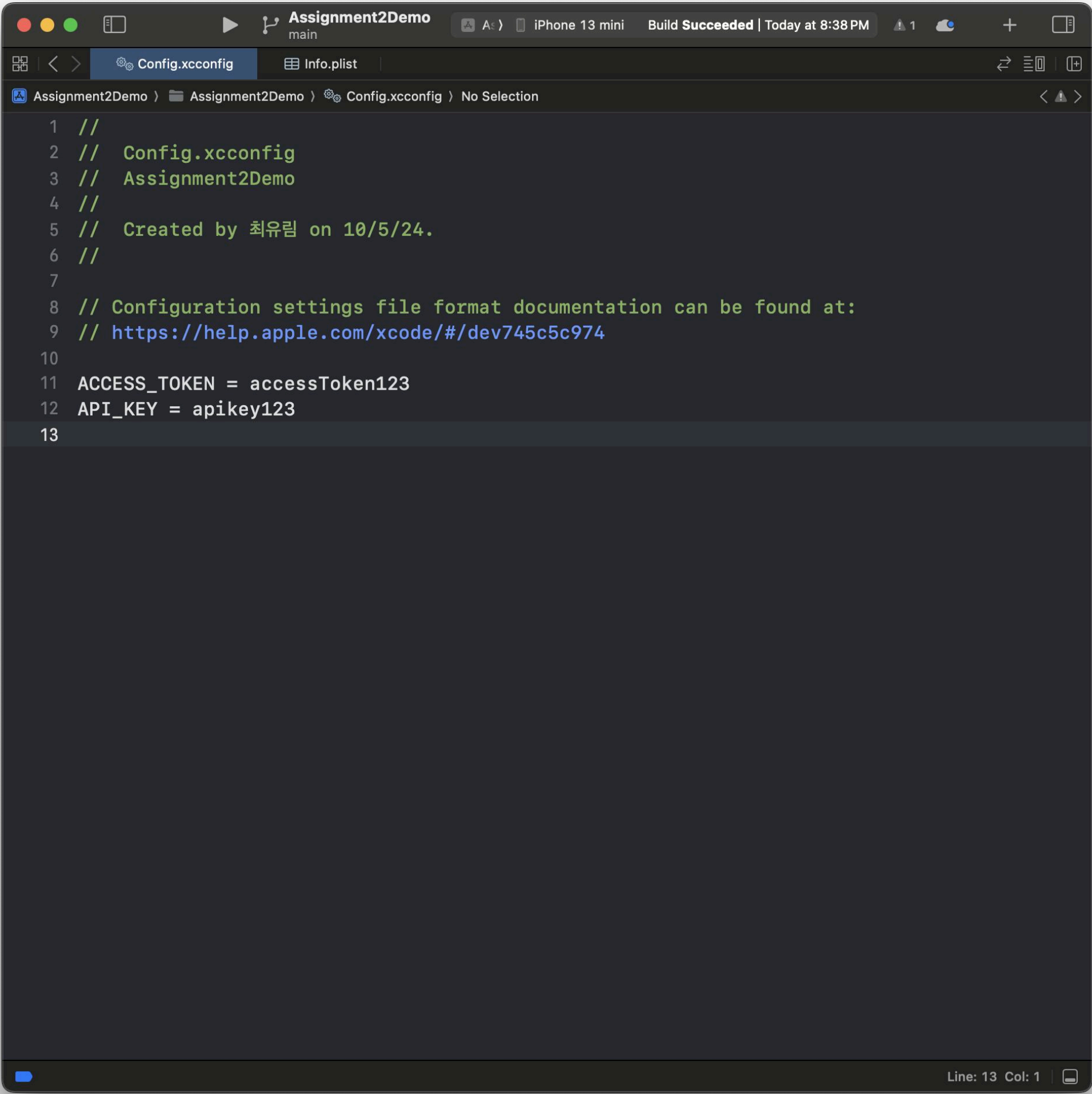
Build Configuration File



Build Configuration File (공식 문서)

- API key, access token 등의 정보
 - 그대로 github의 public repository에 올려버린다면?
- Dev 환경, Prod 환경에 따라 다르게 쓰여야 하는 정보
 - <https://snutt-api-dev.wafflestudio.com/~>
 - <https://snutt-api.wafflestudio.com/~>
 - 똑같은 변수로 다루고 싶은데, 환경에 따라 다른 값이 필요하다면?

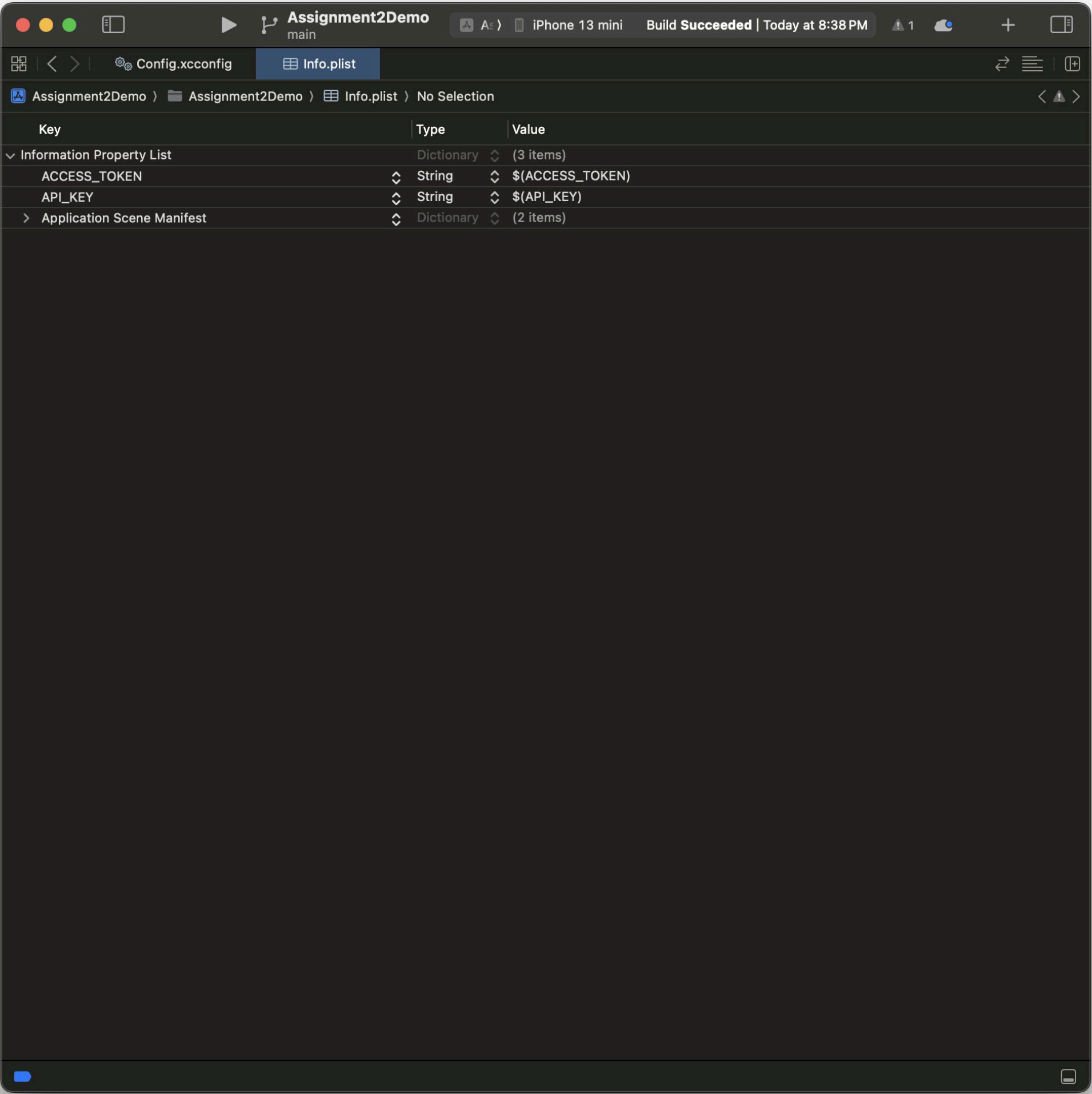
 Build Configuration File



The screenshot shows the Xcode editor with the 'Config.xcconfig' file open. The file contains the following text:

```
1 //  
2 // Config.xcconfig  
3 // Assignment2Demo  
4 //  
5 // Created by 최유림 on 10/5/24.  
6 //  
7  
8 // Configuration settings file format documentation can be found at:  
9 // https://help.apple.com/xcode/#/dev745c5c974  
10  
11 ACCESS_TOKEN = accessToken123  
12 API_KEY = apikey123  
13
```

Config.xcconfig 에서 선언



The screenshot shows the Xcode editor with the 'Info.plist' file open. The file content is displayed in a table format:

Key	Type	Value
Information Property List		
ACCESS_TOKEN	String	\$(ACCESS_TOKEN)
API_KEY	String	\$(API_KEY)
Application Scene Manifest		

Info.plist 에서 등록

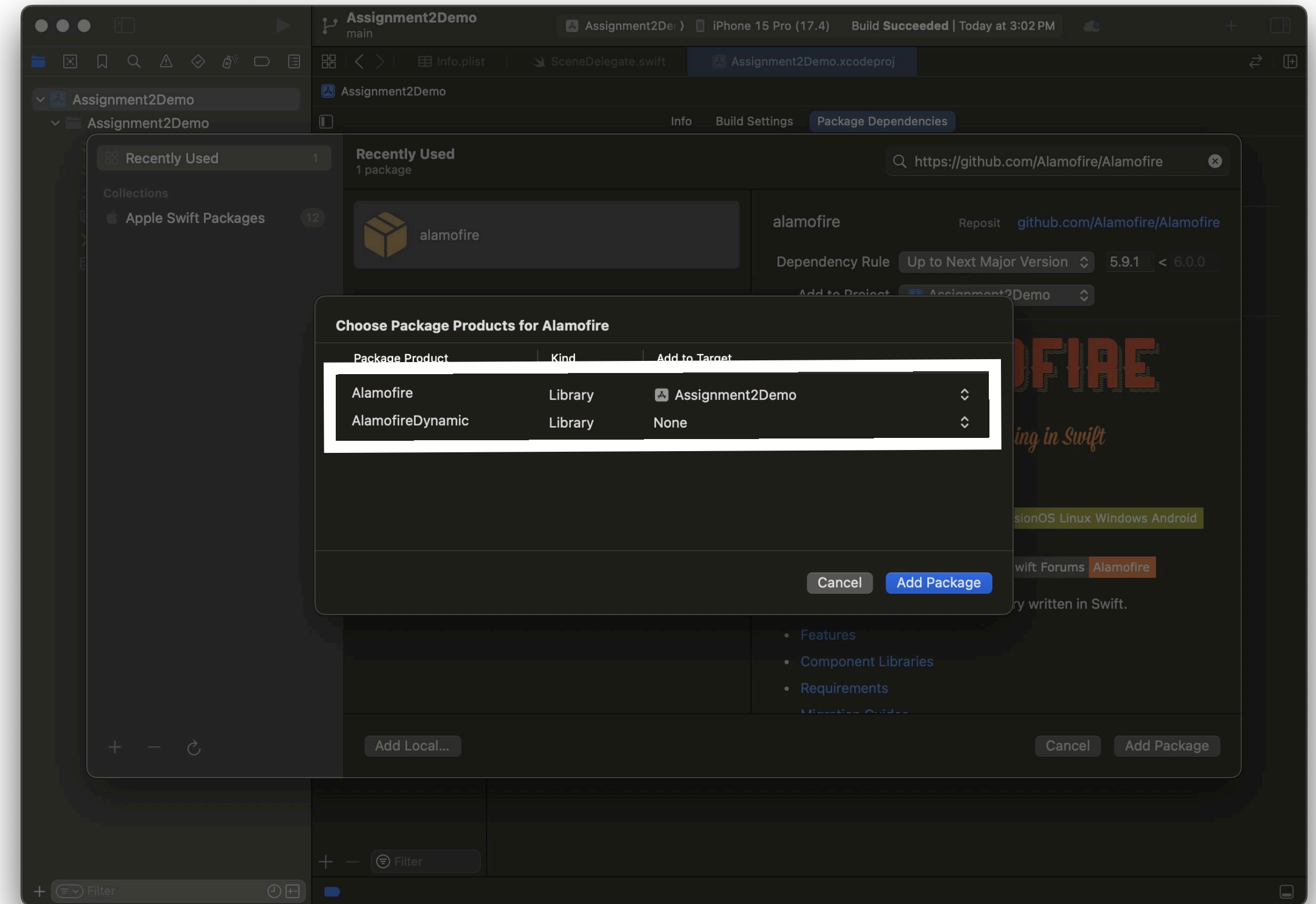
Swift Package Manager

Swift Package Manager (관련 문서)

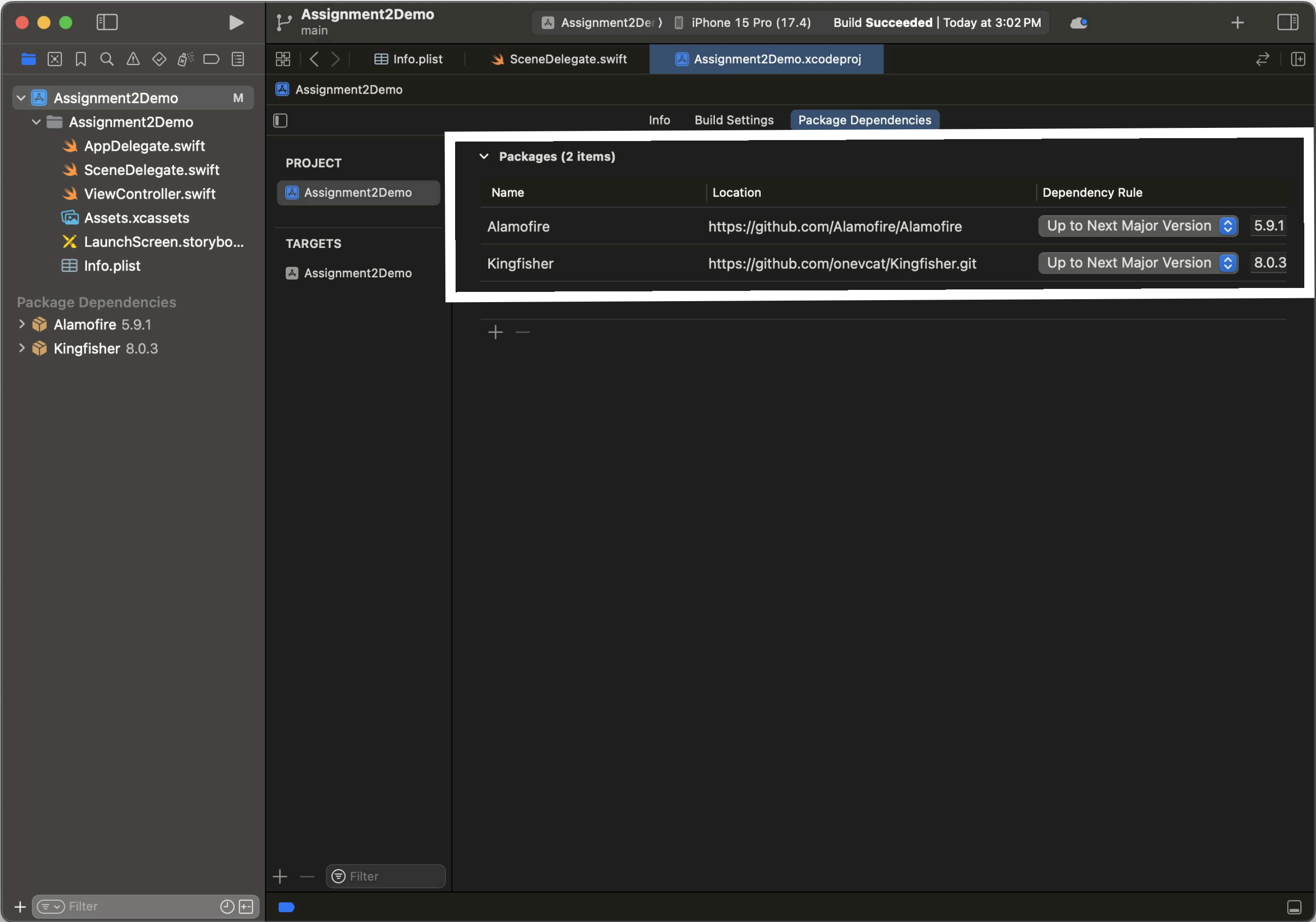
- Dependency 관리를 위한 도구
 - CocoaPods, Carthage 등의 3rd party 도구와 달리 1st party

과제2에서 사용할 Library

- Alamofire
 - <https://github.com/Alamofire/Alamofire>
 - HTTP 네트워킹 라이브러리
 - AF 대신 URLSession을 사용해도 괜찮습니다
- Kingfisher
 - <https://github.com/onevc/Kingfisher.git>
 - 웹에서 비동기로 이미지를 다운받고, 캐싱하는 작업을 도와줌



Alamofire 설정 예시

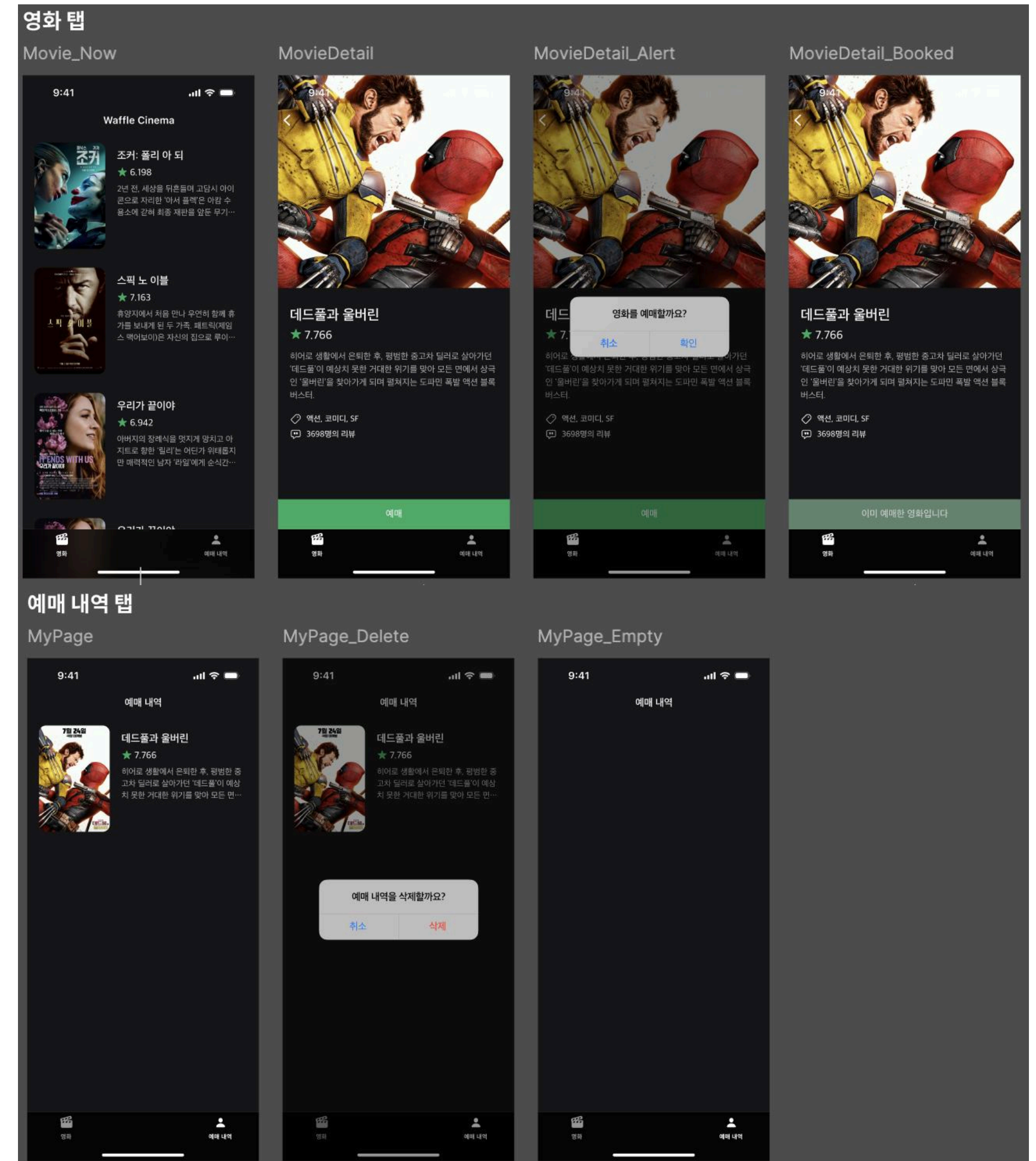


과제2 Package Dependencies 설정 예시

과제 2 안내

간단한 영화 예매 앱 만들기

- 과제 2 디자인 링크(Figma)
- 과제 2 스켈레톤 코드 링크(Github)
- 관련 키워드
 - Package Manager
 - Alamofire
 - Kingfisher
 - UITabBarController
 - UITableView + UITableViewCell
 - UINavigationController
 - UserDefaults
 - UIAlertController

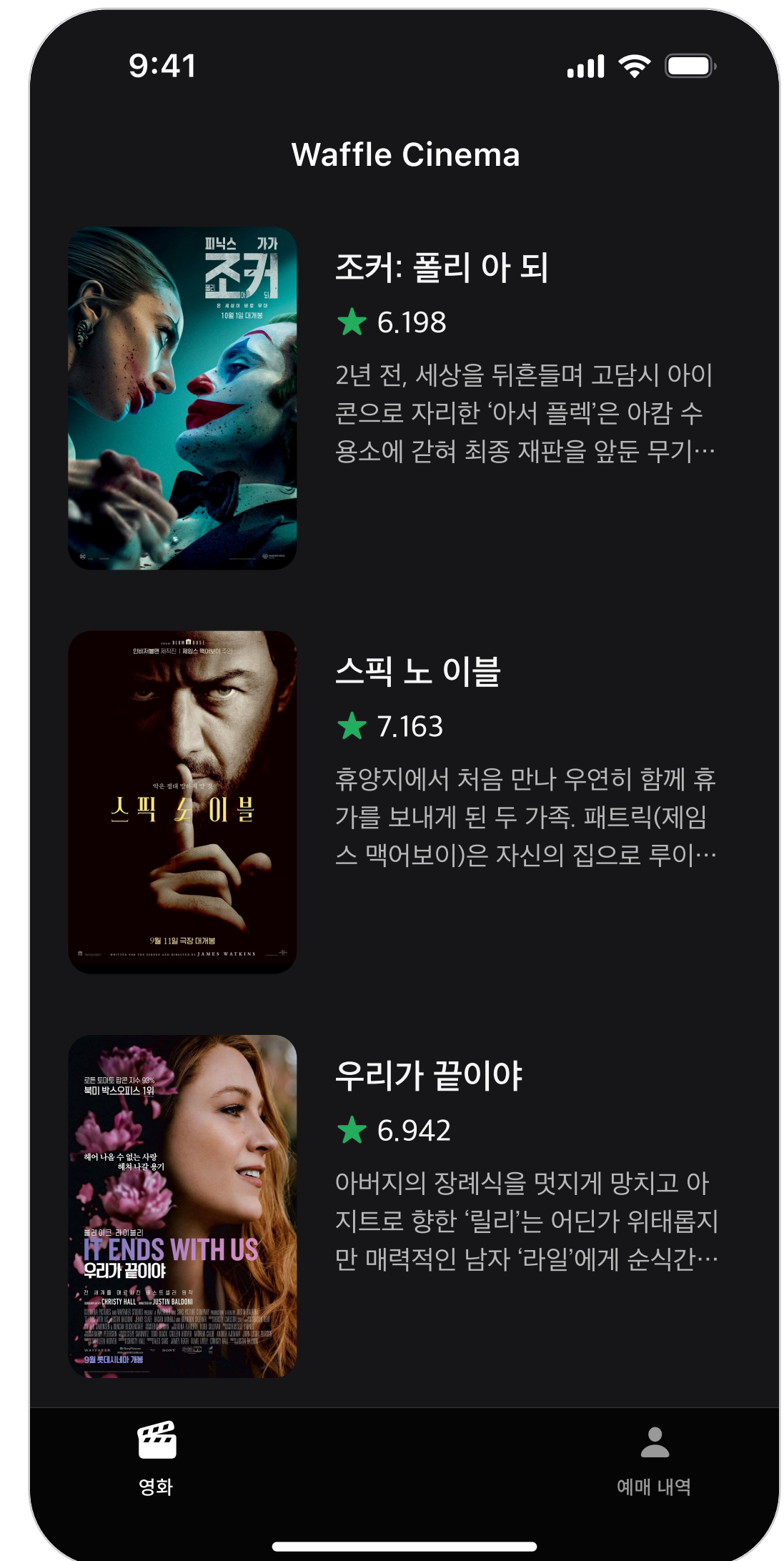


과제 상세 스펙 및 주의사항 (1)

- 전체
 - Storyboard를 사용하지 않습니다 (LaunchScreen.storyboard 제외)
 - 모든 Text, Color, Constraint, Corner Radius 등은 피그마에 주어진 값을 사용합니다
- 스켈레톤 코드
 - 스켈레톤 코드만으로는 과제를 해결하실 수 없습니다. 적절한 변수와 함수를 추가해 주세요
 - 스켈레톤 코드는 자유롭게 수정 가능하며, 아예 사용하지 않으셔도 괜찮습니다
 - 사용 여부는 과제 채점에 아무런 영향을 미치지 않습니다
- Build Configuration File
 - 개별로 발급받은 API key와 Access Token은 **절대로** Github에 올리지 않습니다. Build Configuration File을 이용하고, 세미나장에게 슬랙 DM을 통해 별도로 파일을 전송합니다.
- UIAlertController
 - 별도의 커스텀 디자인이 필요하지 않습니다. 스펙에 맞는 문구만 지정해 주세요

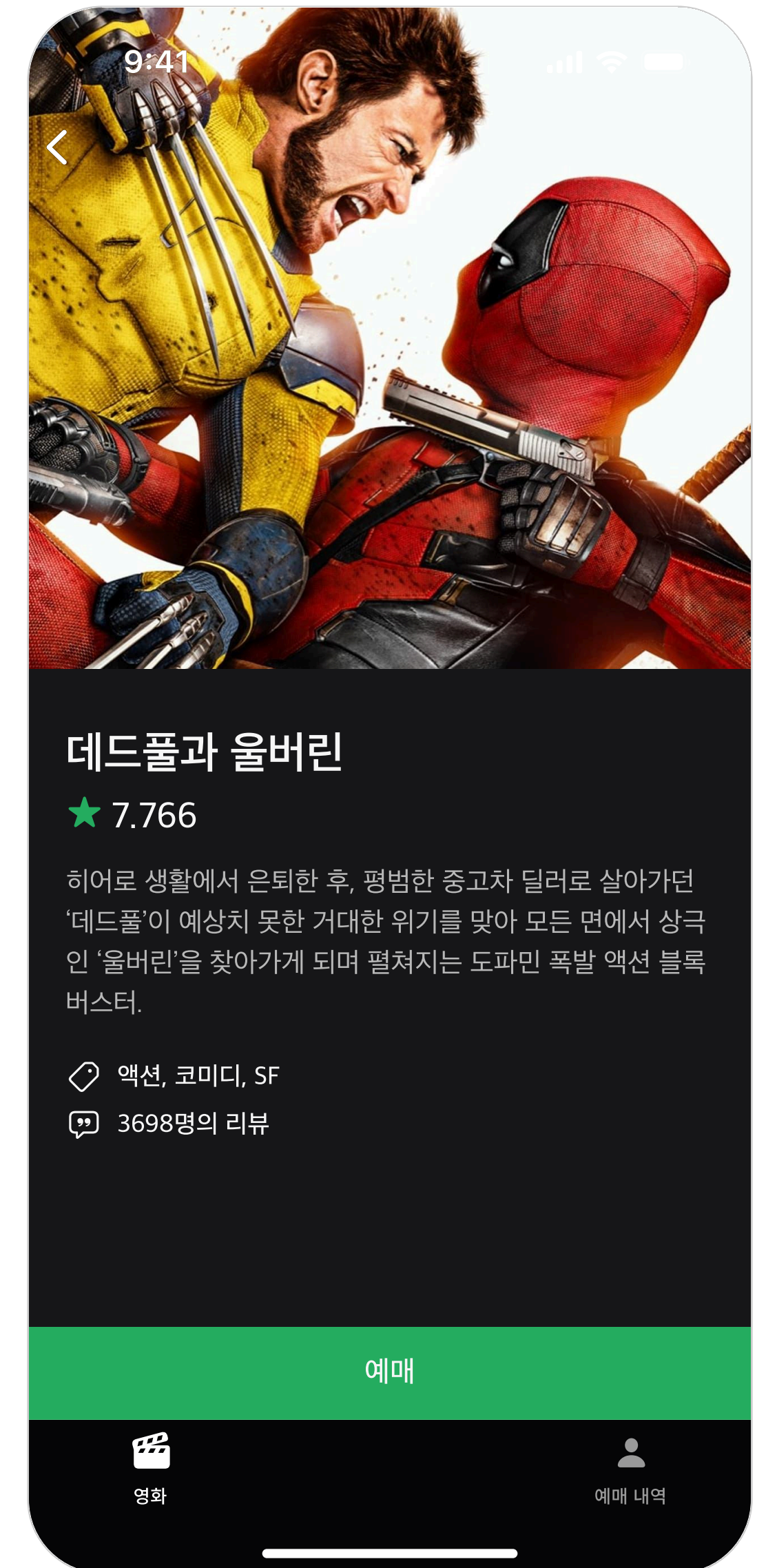
과제 상세 스펙 및 주의사항 (2)

- 영화 탭
 - 영화 목록은 반드시 UITableView를 이용하여 보여줍니다
 - 포스터 이미지는 가로 120, 세로 180으로 고정합니다
 - 포스터 이미지가 없는 경우는 고려하지 않습니다(별도 처리X)
 - 영화의 개요는 최대 3줄까지만 보여줍니다
 - 영화의 평점은 별도의 조작 없이 API response값 그대로 보여줍니다
 - Figma에는 편의상 영화 3개만을 보여주고 있으나, 실제로는 무한 scroll이 가능하도록 합니다
 - Pagination 이용
- 사용할 API



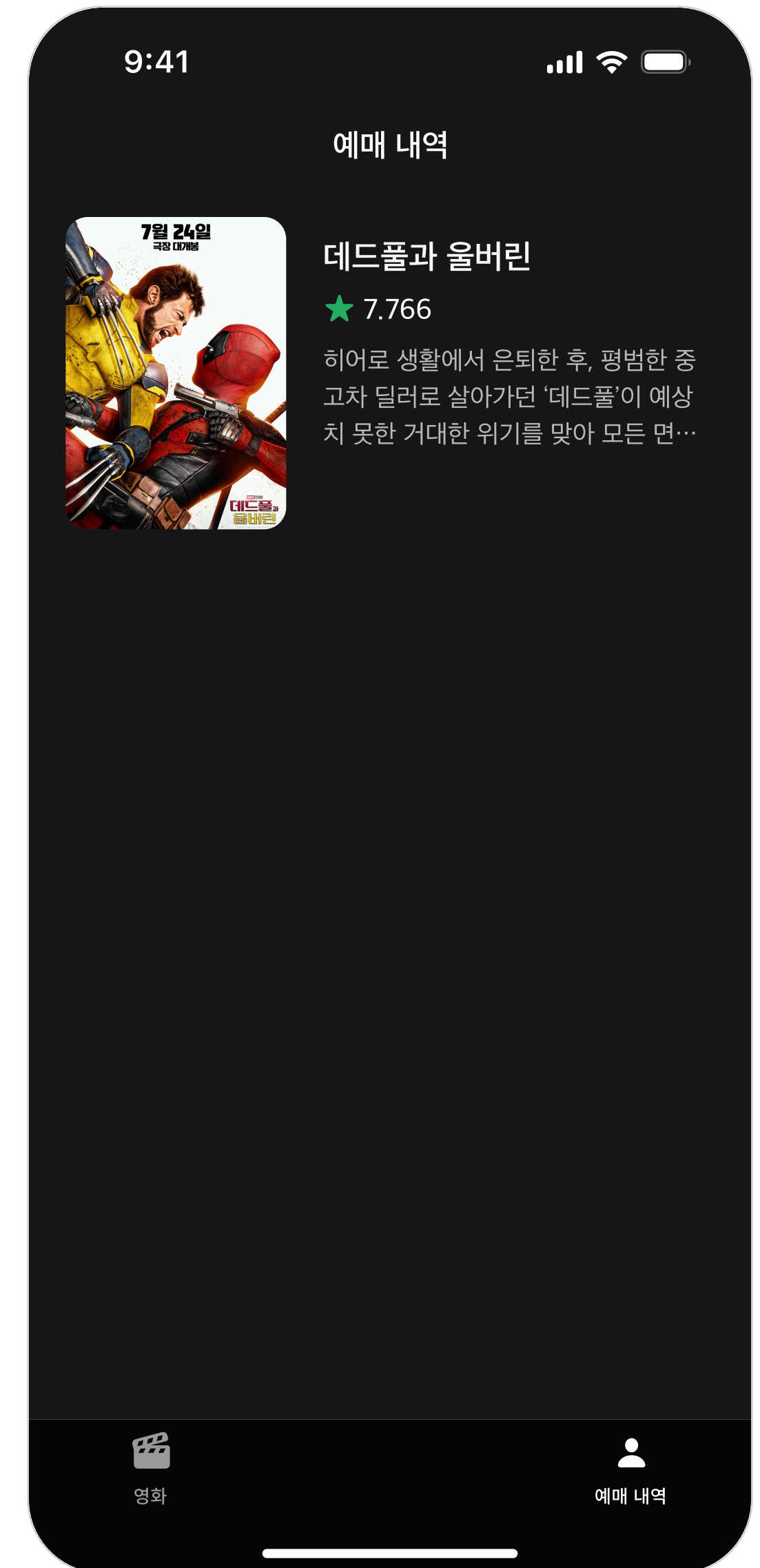
과제 상세 스펙 및 주의사항 (3)

- 영화 상세 화면
 - 포스터 이미지의 가로는 디바이스의 너비와 동일해야 하고, 세로는 360으로 고정합니다
 - 예매 버튼은 화면 아래에 고정하고, 그 외 정보는 스크롤뷰로 보여줍니다
 - 자세한 동작은 과제 Demo 영상 참고
 - hint: UIScrollView
 - 이미 예매한 영화라면 예매 버튼을 비활성화합니다
 - 비활성화 상태의 버튼 문구와 색상에 유의합니다
 - 자세한 동작은 과제 Demo 영상 참고
- 사용할 API

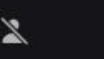


과제 상세 스펙 및 주의사항 (4)

- 예매 내역 탭
 - 영화 목록은 반드시 UITableView를 이용하여 보여줍니다
 - cell의 스펙은 “영화” 탭과 동일
 - cell을 누르면, UIAlertController를 띄워 예매 내역을 지울지 확인합니다
 - 예매 내역은 앱을 종료 후 다시 실행하는 경우에도 유지됩니다
 - hint: UserDefaults



과제 Bonus 스펙

- 영화 상세 화면의 포스터 이미지에 그라데이션을 추가합니다
- 영화의 출연진 정보를 추가합니다
 - Figma는 일부 출연진만을 보여주고 있지만, 실제로는 모든 출연진을 보여줍니다
 - 출연진은 반드시 UICollectionView를 이용하여 보여줍니다
 - cell의 width: 디바이스 크기에 따라 변합니다
 - cell의 height: 편의상 232로 고정하고, 텍스트가 길어 넘치는 경우는 고려하지 않습니다
 - 출연진 프로필 이미지는 가로 100, 세로 150으로 고정합니다
 - 이미지가 없는 경우에는 Figma에 있는 이미지()를 보여줍니다
- 사용할 API

